Website QA Report

Task 1 – Automation & QA Developer Take-Home
Tester: Darshan | Date: 10 Nov 2026

1. Scope and test environment

> **Important disclosure.** The public demo `https://demo.realworld.io/` could not be reached from the environment where this testing was executed (every request to it, and to other hosted RealWorld demos, failed at network level). The assignment allows "a suitable working RealWorld implementation", so all testing below was run against a **self-hosted copy** of a public RealWorld ("Conduit") implementation. Every bug was found by running the tests described here; none are copied from the repository's issue tracker. Results apply to this implementation and may differ on the hosted demo.

Item	Detail
Application under test	TonyMckes/conduit-realworld-example-app (React + Vite frontend, Express + Sequelize + PostgreSQL 16 backend), commit `5e127d8`
URLs	Frontend `http://localhost:3000/#/` (hash routing), API `http://localhost:3001/api`
Browser / tools	Chromium (Playwright) driving the real UI; `curl`/fetch for API-level checks; `psql` to inspect the database
Method	Black-box UI testing for each flow (sign-up, login, create, edit, delete, logout, comments, settings), then API probing for validation, authorization and error handling. Each scenario was run on a clean database or with unique accounts, and failures were re-run in isolation before being reported. Source code was read only afterwards, to explain the suspected cause.
Not covered	Hosted demo, other browsers, mobile layout, accessibility audit, load/performance, favorites/follow UI flows (API only).

2. Summary

10 genuine defects were found: 1 Critical, 3 High, 4 Medium, 2 Low. Authorization checks (edit/delete by another user, unauthenticated calls) and output escaping behaved correctly.

ID	Severity	Title	Area
BUG-01	Critical	Saving the Settings page silently overwrites the user's password; an EMPTY password then logs in	Settings / Auth
BUG-02	High	Changing the e-mail address breaks the session: every later action fails with "User not found"	Settings / Session
BUG-03	High	Duplicate usernames are accepted (sign-up and Settings)	Sign-up / Settings
BUG-04	High	Duplicate e-mails accepted via Settings and via different letter case at sign-up	Settings / Sign-up
BUG-05	Medium	Weak passwords accepted at sign-up; sign-up and login rules disagree, so the account cannot log in through the UI	Sign-up / Login validation
BUG-06	Medium	Whitespace-only input accepted; an article with an empty slug is created and cannot be opened, edited or deleted	Create article / Sign-up
BUG-07	Medium	Over-long input and malformed requests return HTTP 500 and expose internal error text	Error handling
BUG-08	Medium	Tags with 1-2 characters (e.g. "qa", "js") are silently dropped	Create/Edit article

BUG-09	**Low**	Non-Latin titles produce meaningless slugs and false "Title already exists" errors	Create article
BUG-10	**Low**	Login errors reveal whether an e-mail is registered (account enumeration)	Login / Security

Severity: Critical = security/data compromise; High = core function broken or data integrity violated; Medium = wrong behaviour with a workaround; Low = cosmetic or hardening.

3. Test coverage

Test	Result
Sign-up: valid user	Pass
Sign-up: empty fields (browser `required`)	Pass (client-side only)
Sign-up: invalid e-mail `abc`	Pass (blocked by browser)
Sign-up: duplicate e-mail (exact)	Pass (422 shown)
Sign-up: duplicate username	**Fail** BUG-03
Sign-up: weak / 1-char password	**Fail** BUG-05
Sign-up: 300-char username	**Fail** BUG-07
Sign-up: whitespace-only username	**Fail** BUG-06
Sign-up: e-mail differing by case	**Fail** BUG-04
Login: valid credentials	Pass
Login: wrong password / unknown e-mail	Pass (rejected) / **Fail** BUG-10 (message leaks)
Login: empty fields	Pass (browser validation, no request sent)
Session persists after page reload	Pass
Logout clears stored user and navbar	Pass
Protected routes (`#/editor`, `#/settings`) when logged out	Pass (redirect to home)
Create article: valid, with markdown	Pass
Create article: empty fields	Pass (browser validation)
Create article: whitespace-only	**Fail** BUG-06
Create article: duplicate title	Pass (422 shown; copy has a trailing "..")
Create article: 300-char title	**Fail** BUG-07
Create article: HTML/script in title, description, body, comment	Pass (escaped; no script ran)
Create article: unicode / emoji title	Pass (renders) / **Fail** BUG-09 (slug)
Create article: tags 1-2 chars	**Fail** BUG-08
Edit own article (title/body), tags prefilled	Pass (note: URL slug changes with title)
Edit/Delete article as another user via API	Pass (403)
Edit/Delete without token via API	Pass (401)
Delete own article (confirm dialog, redirect, URL then 404)	Pass
Comment: add, empty rejected, delete by non-author (403)	Pass
Settings: update bio	**Fail** BUG-01 (side effect)
Settings: change e-mail	**Fail** BUG-02, BUG-04
Settings: change username to an existing one	**Fail** BUG-03
Non-existent article / profile / route	Pass (404 page)
Pagination (`limit` / `offset`) valid values	Pass
Pagination invalid values	**Fail** BUG-07

4. Detailed bug reports

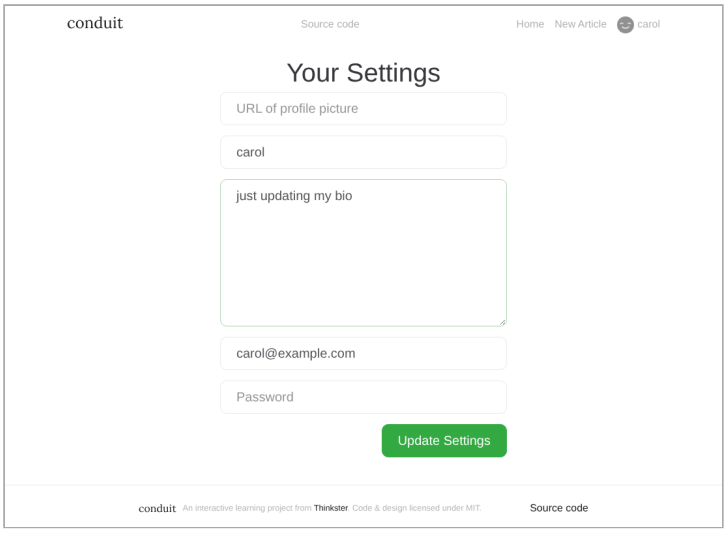
BUG-01 — Saving the Settings page silently overwrites the user's password; an EMPTY password then logs in

Critical Area: Settings / Auth

Steps to reproduce

1. Open `http://localhost:3000/#/register` and create user `carol` / `carol@example.com` / `Passw0rd!`.
2. Open Settings (`#/settings`). Change only the Bio field. Leave the Password field blank. Click **Update Settings**.
3. Log out. Try to log in with `Passw0rd!` (the original password).
4. Send `POST /api/users/login` with `{"user":{"email":"carol@example.com","password":""}}` (the UI login form blocks passwords shorter than 5 characters, so use curl/Postman).

Expected: Password is unchanged when the Password field is left blank. Original password still works; an empty password is rejected.
Actual: HTTP 200 on save. Afterwards the original password returns 422 "Wrong email/password combination" (user locked out) and an **empty** password returns 200 with a valid token (anyone who knows the email can sign in).
Suspected cause: `backend/controllers/user.js` uses `if (password !== undefined || password !== "")`. This condition is always true, so a blank password from the Settings form is bcrypt-hashed and stored. The condition should use `&&`.



Only the Bio is changed, Password left blank *(29_settings_bio_only.png)*

```
BUG-01 Saving Settings overwrites the password (live API calls)

# 1. Sign up a user (password: Passw0rd!)
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"ev1947","email":"ev1947@example.com","password":"Passw0rd!"}}'
{"user":{"email":"ev1947@example.com","username":"ev1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# 2. Login with the real password -> works
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users/login -H 'Content-Type: application/json'  -d '{"use
r":{"email":"ev1947@example.com","password":"Passw0rd!"}}'
{"user":{"email":"ev1947@example.com","username":"ev1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 200]

# 3. Login with an EMPTY password -> rejected (expected)
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users/login -H 'Content-Type: application/json'  -d '{"use
r":{"email":"ev1947@example.com","password":""}}'
{"errors":{"body":["Wrong email/password combination"]}} [HTTP 422]

# 4. Same request the Settings page sends when only the bio is changed (password field left blank)
$ curl -s -w ' [HTTP %{http_code}]' -X PUT http://127.0.0.1:3001/api/user -H 'Content-Type: application/json' -H 'Authorization:
Token <JWT>' -d '{"user":{"bio":"only changing my bio","password":""}}'
{"user":{"username":"ev1947","bio":"only changing my bio","image":null,"token":"<JWT>"}} [HTTP 200]

# 5. Login with the REAL password again
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users/login -H 'Content-Type: application/json'  -d '{"use
r":{"email":"ev1947@example.com","password":"Passw0rd!"}}'
{"errors":{"body":["Wrong email/password combination"]}} [HTTP 422]

# 6. Login with an EMPTY password
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users/login -H 'Content-Type: application/json'  -d '{"use
r":{"email":"ev1947@example.com","password":""}}'
{"user":{"email":"ev1947@example.com","username":"ev1947","bio":"only changing my bio","image":null,"token":"<JWT>"}} [HTTP 200]
```

Live API evidence: original password rejected, empty password accepted after the save *(40_evidence_password_overwrite.png)*

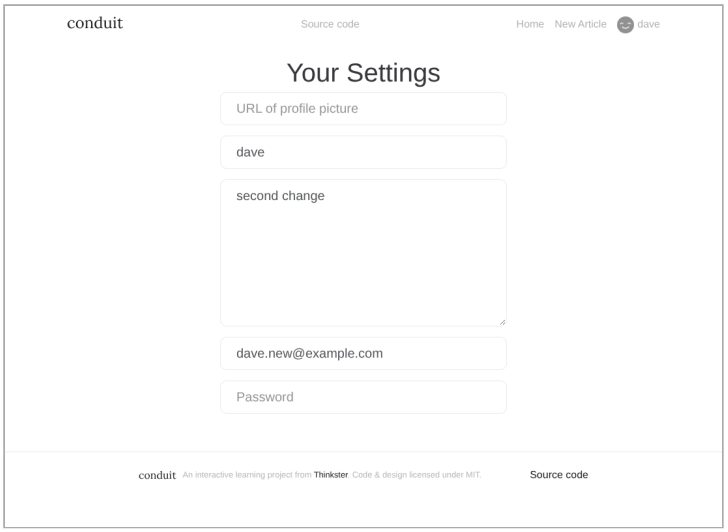
BUG-02 — Changing the e-mail address breaks the session: every later action fails with "User not found"

High Area: Settings / Session

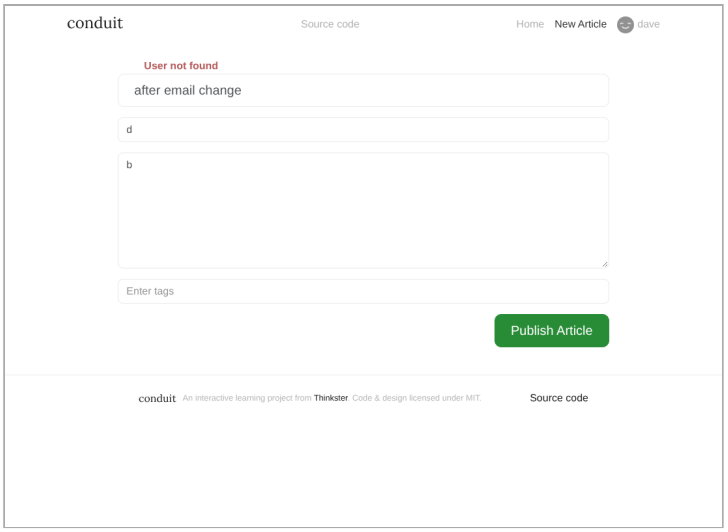
Steps to reproduce

1. Register and log in as any user.
2. Open Settings, change the Email to a new unique address, click **Update Settings** (succeeds).
3. Without logging out, change the Bio and save again; then try to publish a new article.

Expected: Both later actions succeed (or the app issues a fresh token for the new e-mail).
Actual: The 2nd Settings save returns 404 "User not found" and the Settings page shows **no error at all** (the edit is silently lost); publishing an article fails with the same red error and the form is not submitted. The navbar still shows the user as logged in. Only logout + login fixes it.
Suspected cause: The JWT identifies the user by e-mail and the backend looks the user up by the e-mail inside the token (`middleware/verifyToken.js`). The update endpoint returns the *old* token after changing the e-mail, so the next request looks up an e-mail that no longer exists.



Settings page after the 2nd save: bio shows as saved but no error is displayed (API returned 404) *(27_settings_after_email_change.png)*



Publishing an article after the e-mail change *(28_publish_after_email_change.png)*

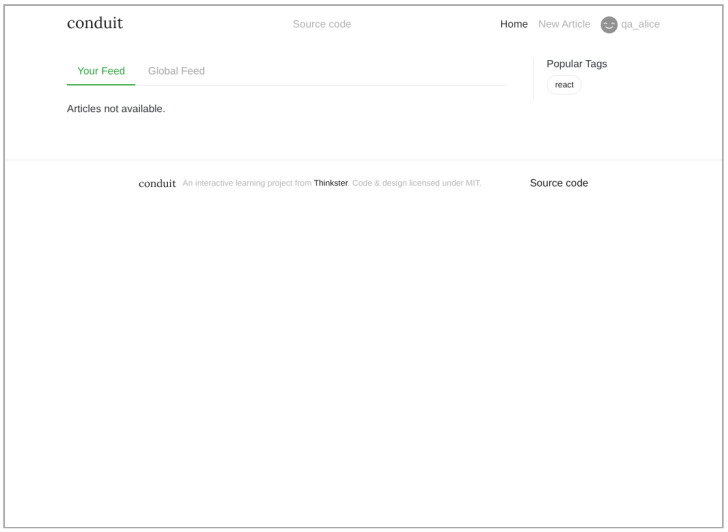
BUG-03 — Duplicate usernames are accepted (sign-up and Settings)

High Area: Sign-up / Settings

Steps to reproduce

1. Register `qa_alice` with `alice@example.com`.
2. Log out. Register again with username `qa_alice` and a different e-mail `alice2@example.com`.
3. (Optional) In Settings of another user, change the username to an existing one and save.

Expected: Second sign-up is rejected ("username already taken"), as the RealWorld spec requires unique usernames.
Actual: Second account is created (HTTP 201) and the user is logged in. Two users now share one username, which is the key used in profile URLs (`#/profile/<username>`) and author links. Settings also accepts renaming to an existing username (HTTP 200).
Suspected cause: `controllers/users.js` only checks for an existing *e-mail*; the `User` model has no `unique` constraint on `username`.



Duplicate username: sign-up succeeded and redirected to home *(07_signup_duplicate_username.png)*

```
BUG-02/03 Duplicate usernames and case-variant emails accepted (live API calls)

# Create user "dupname" (email 1)
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"dupname1947","email":"d1_1947@example.com","password":"Passw0rd!"}}'
{"user":{"email":"d1_1947@example.com","username":"dupname1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# Create a SECOND user with the same username
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"dupname1947","email":"d2_1947@example.com","password":"Passw0rd!"}}'
{"user":{"email":"d2_1947@example.com","username":"dupname1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# Same email, different letter case
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"dupcase1947","email":"D1_1947@EXAMPLE.COM","password":"Passw0rd!"}}'
{"user":{"email":"D1_1947@EXAMPLE.COM","username":"dupcase1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# DB check: rows for the username
$ psql -h 127.0.0.1 -p 5433 -U postgres conduit -c "select id,username,email from \"Users\" where username='dupname1947' or lowe
r(email)='d1_1947@example.com' order by id"
id |  username   |        email
----+-------------+---------------------
 29 | dupname1947 | d1_1947@example.com
 30 | dupname1947 | d2_1947@example.com
 31 | dupcase1947 | D1_1947@EXAMPLE.COM
(3 rows)
```

Live API/DB evidence *(41_evidence_duplicate_accounts.png)*

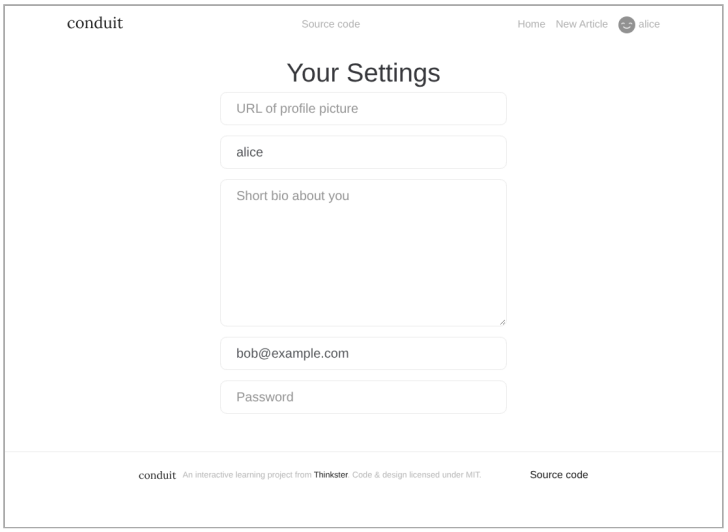
BUG-04 — Duplicate e-mails accepted via Settings and via different letter case at sign-up

High Area: Settings / Sign-up

Steps to reproduce

1. Create users `alice@example.com` and `bob@example.com`.
2. Log in as alice, open Settings, set Email to `bob@example.com`, save.
3. Also try signing up with `ALICE@EXAMPLE.COM`.

Expected: Both are rejected with an "e-mail already exists" error.
Actual: Settings returns 200 and the database now has two users with `bob@example.com`; a login for that e-mail returned only one of them (`bob`). `ALICE@EXAMPLE.COM` sign-up returns 201 (second account for the same mailbox).
Suspected cause: Uniqueness is checked only at sign-up with a case-sensitive exact match; `updateUser` has no check at all and the column has no unique index.



Settings accepted an e-mail already used by another account *(26_settings_email_taken_by_other_user.png)*

```
BUG-02/03 Duplicate usernames and case-variant emails accepted (live API calls)

# Create user "dupname" (email 1)
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"dupname1947","email":"d1_1947@example.com","password":"Passw0rd!"}}'
{"user":{"email":"d1_1947@example.com","username":"dupname1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# Create a SECOND user with the same username
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"dupname1947","email":"d2_1947@example.com","password":"Passw0rd!"}}'
{"user":{"email":"d2_1947@example.com","username":"dupname1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# Same email, different letter case
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/users -H 'Content-Type: application/json'  -d '{"user":{"u
sername":"dupcase1947","email":"D1_1947@EXAMPLE.COM","password":"Passw0rd!"}}'
{"user":{"email":"D1_1947@EXAMPLE.COM","username":"dupcase1947","bio":null,"image":null,"token":"<JWT>"}} [HTTP 201]

# DB check: rows for the username
$ psql -h 127.0.0.1 -p 5433 -U postgres conduit -c "select id,username,email from \"Users\" where username='dupname1947' or lowe
r(email)='d1_1947@example.com' order by id"
id |  username   |        email
----+-------------+---------------------
 29 | dupname1947 | d1_1947@example.com
 30 | dupname1947 | d2_1947@example.com
 31 | dupcase1947 | D1_1947@EXAMPLE.COM
(3 rows)
```

DB rows showing the duplicates *(41_evidence_duplicate_accounts.png)*

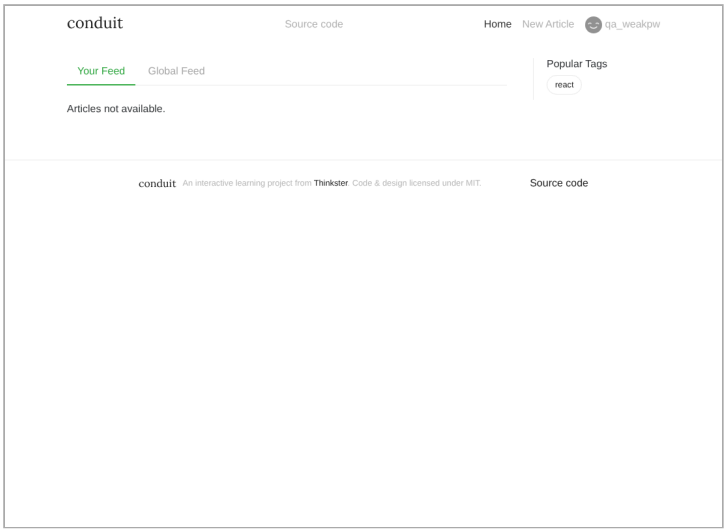
BUG-05 — Weak passwords accepted at sign-up; sign-up and login rules disagree, so the account cannot log in through the UI

Medium Area: Sign-up / Login validation

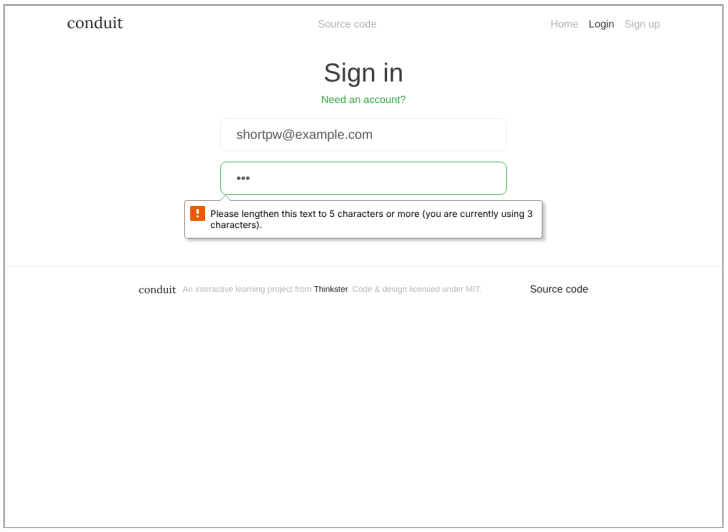
Steps to reproduce

1. Open `#/register`, sign up with password `abc` (or a single character).
2. Log out, open `#/login`, enter the same e-mail and `abc`, click Login.

Expected: Either sign-up rejects the weak password, or login accepts whatever sign-up accepted.
Actual: Sign-up succeeds with a 1-character password. The login form shows the browser message "Please lengthen this text to 5 characters or more" and sends no request, so the user can never sign in from the UI (the API accepts the same credentials with 200).
Suspected cause: The login input has `minlength=5` but the sign-up input and the backend have no length or strength rule.



Sign-up accepted a 1-character password *(08_signup_one_char_password.png)*



Login form blocks the same password *(08b_short_password_cannot_login.png)*

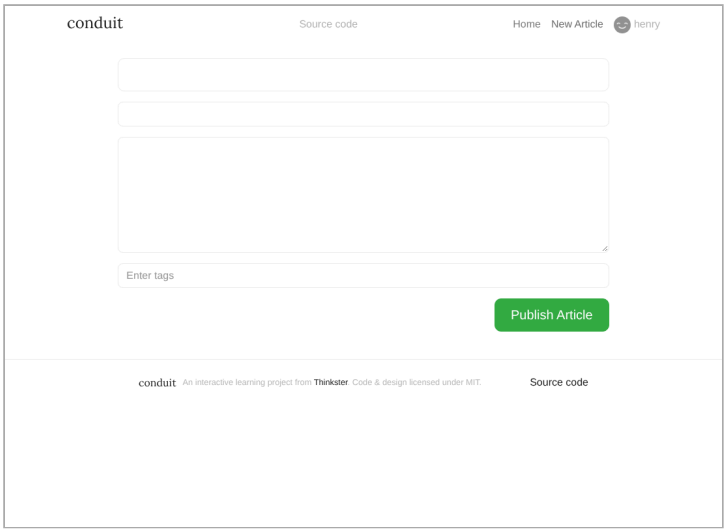
BUG-06 — Whitespace-only input accepted; an article with an empty slug is created and cannot be opened, edited or deleted

Medium Area: Create article / Sign-up

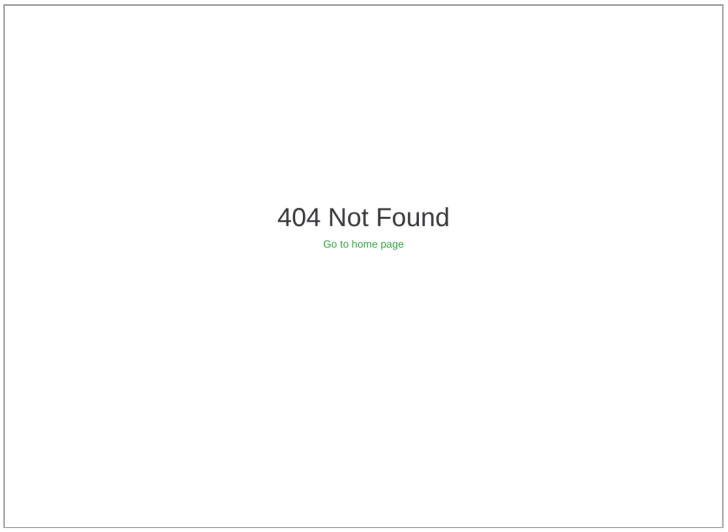
Steps to reproduce

1. Log in. Open `#/editor`.
2. Type three spaces in Title, Description and Body. Click **Publish Article**.
3. Open Global Feed and click the blank entry.

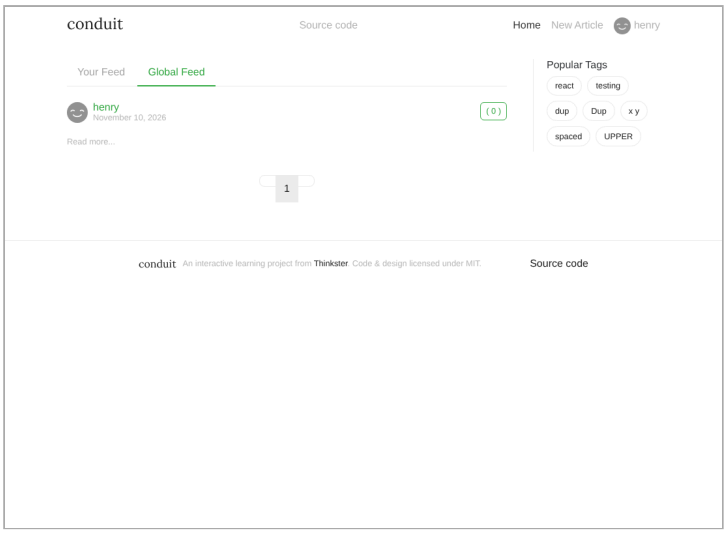
Expected: Validation error ("title can't be blank"). No article is created.
Actual: HTTP 201 and the app navigates to `#/article/` which shows "404 Not Found". A blank entry stays in the Global Feed; its link is `#/article/`, and PUT/DELETE on `/api/articles/` return 404, so the author cannot remove it from the UI. Sign-up also accepts a whitespace-only username (HTTP 201).
Suspected cause: The browser `required` attribute and the server check (`!username`) only reject empty strings, not whitespace. `slugify()` trims the title, producing the slug `""`.



Whitespace-only form *(12a_whitespace_only_form.png)*



Result of Publish: 404 page *(12_article_whitespace_only.png)*



Blank entry in Global Feed *(35_home_feed_blank_article.png)*

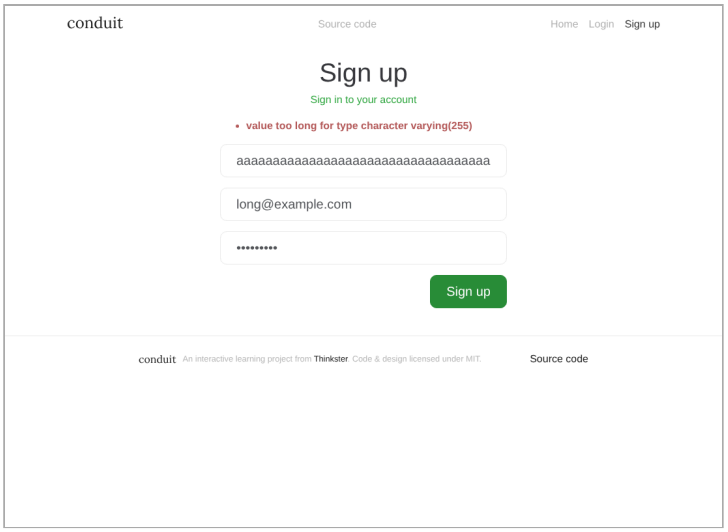
BUG-07 — Over-long input and malformed requests return HTTP 500 and expose internal error text

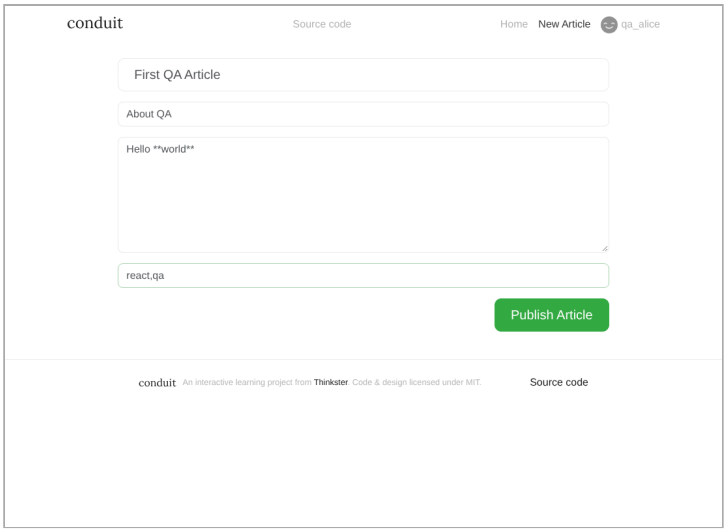
Medium Area: Error handling

Steps to reproduce

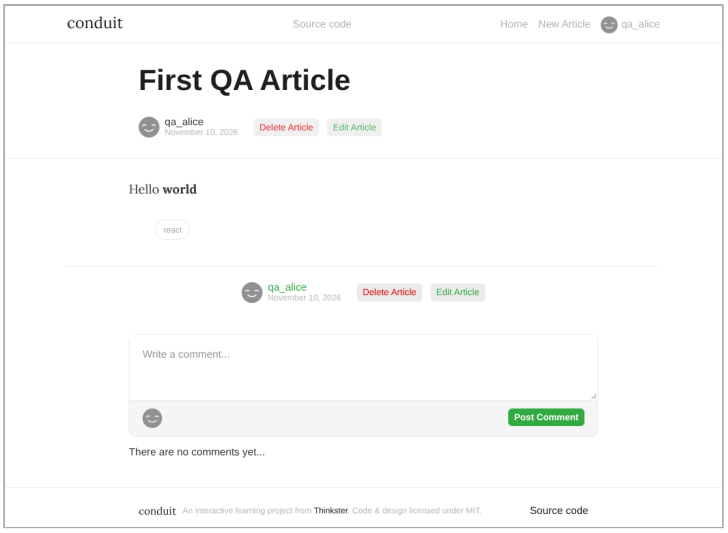
1. Sign up with a 300-character username (UI: `#/register`), or publish an article with a 300-character title.
2. API: `GET /api/articles?limit=-1`, `?limit=abc`, `GET /api/user` with header `Authorization: Token garbage`, and `POST /api/users/login` with body `{bad json`.

Expected: 4xx responses (400/401/422) with a user-friendly message such as "Username must be at most 50 characters".
Actual: All return HTTP 500. The UI prints raw database text: *value too long for type character varying(255)*. Other responses leak internals: *column "nan" does not exist*, *jwt malformed*, *LIMIT must not be negative*.
Suspected cause: No input-length or query-parameter validation; `middleware/errorHandler.js` turns every unknown exception into 500 and returns `error.message` to the client.


conduit
Source code
Home
Login
Sign up
Sign up
Sign in to your account
value too long for type character varying(255)
aaaa



Tags typed in the editor *(10_editor_filled.png)*



Article page: only 'react' remains *(11_article_created.png)*

```
BUG-08 Tags of 1-2 characters are silently dropped (live API calls)

# Create article with tags qa, js, react, ab
$ curl -s -w ' [HTTP %{http_code}]' -X POST http://127.0.0.1:3001/api/articles -H 'Content-Type: application/json' -H 'Authoriz
ation: Token <JWT>' -d '{"article":{"title":"tag test 1947","description":"d","body":"b","tagList":["qa","js","react","ab"]}}'
{"article":{"slug":"tag-test-1947","title":"tag test 1947","description":"d","body":"b","updatedAt":"2026-11-10T01:53:23.603
Z","createdAt":"2026-11-10T01:53:23.596Z","tagList":["qa","js","react","ab"],"author":{"username":"ev1947","bio":"only changing
my bio","image":null,"following":false,"followersCount":0},"favorited":false,"favoritesCount":0}} [HTTP 201]

# Fetch the same article
$ curl -s -w ' [HTTP %{http_code}]' http://127.0.0.1:3001/api/articles/tag-test-1947
{"article":{"slug":"tag-test-1947","title":"tag test 1947","description":"d","body":"b","createdAt":"2026-11-10T01:53:23.596
Z","updatedAt":"2026-11-10T01:53:23.603Z","tagList":["react"],"author":{"username":"ev1947","bio":"only changing my bio","imag
e":null,"following":false,"followersCount":0},"favorited":false,"favoritesCount":0}} [HTTP 200]
```

API: create response vs stored tags *(43_evidence_tags_dropped.png)*

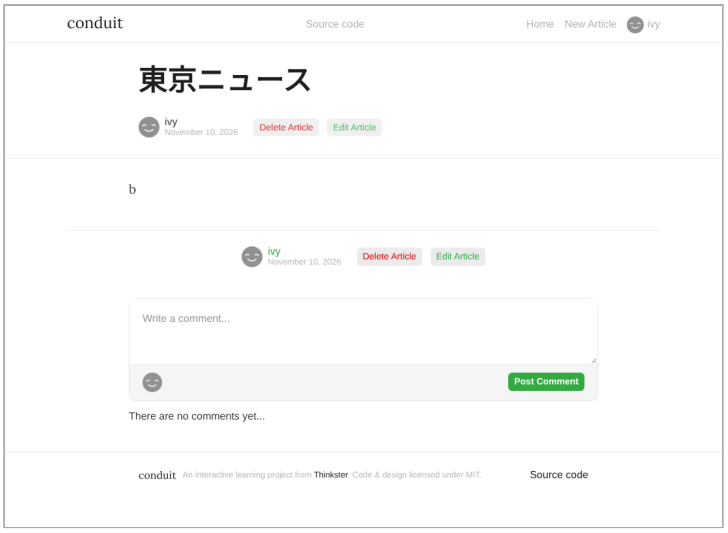
BUG-09 — Non-Latin titles produce meaningless slugs and false "Title already exists" errors

Low Area: Create article

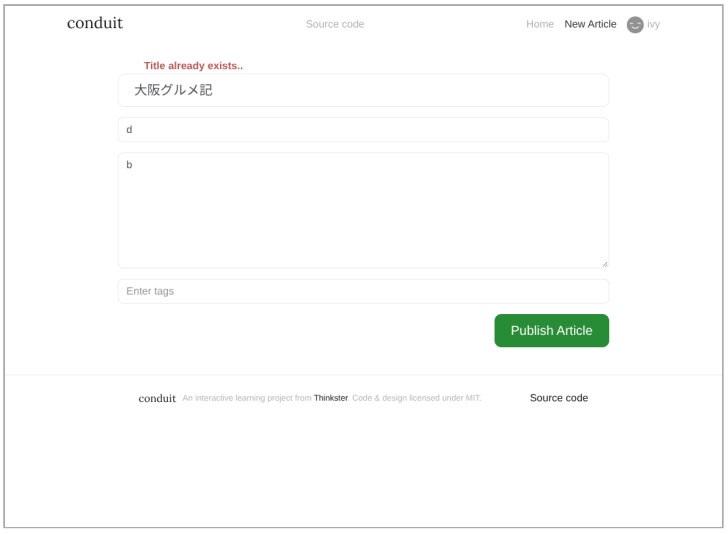
Steps to reproduce

1. Publish an article titled `東京ニュース`.
2. Publish another article titled `大阪グルメ記`.

Expected: Both articles are created with distinct, readable URLs.
Actual: The first gets the URL `#/article/------`. The second, a different title of the same length, is rejected with "Title already exists..". Titles such as `Español: ¿Qué tal?` become `espa-ol---qu--tal-`.
Suspected cause: `helpers.js` slugify is `trim().toLowerCase().replace(/\W|_/g, "-")`; `\W` matches every non-ASCII character, and uniqueness is enforced on the slug rather than a separate unique id.



First Japanese title: URL is only dashes *(36a_japanese_title_slug_dashes.png)*



Second, different title is rejected *(36_unicode_title_collision.png)*

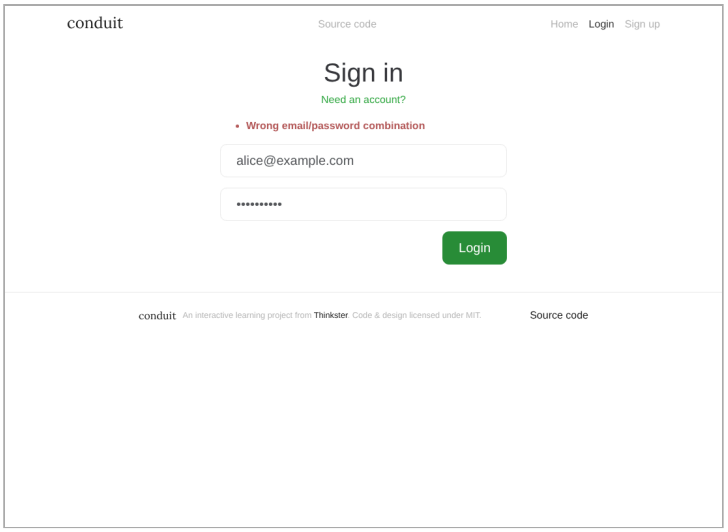
BUG-10 — Login errors reveal whether an e-mail is registered (account enumeration)

Low Area: Login / Security

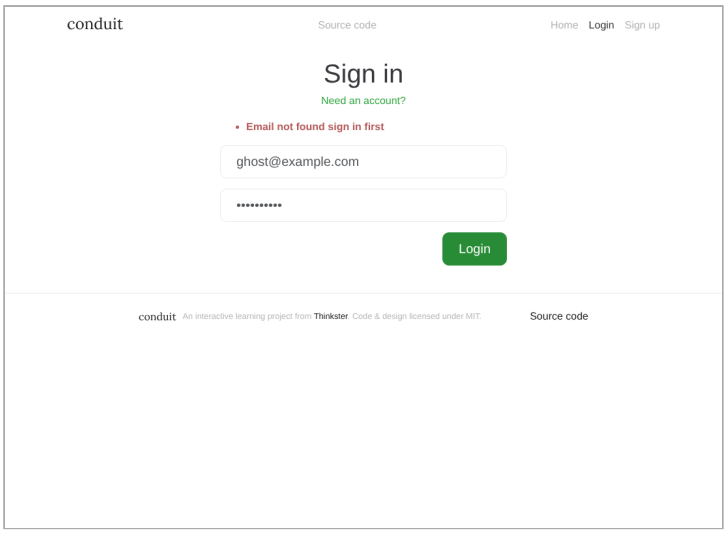
Steps to reproduce

1. On `#/login` enter a registered e-mail with a wrong password (6+ characters).
2. Repeat with an unregistered e-mail.

Expected: Same generic message in both cases, e.g. "Invalid e-mail or password".
Actual: Registered e-mail: 422 "Wrong email/password combination". Unregistered: 404 "Email not found sign in first". Sign-up also says "Email already exists". An attacker can build a list of valid accounts.
Suspected cause: `signIn` in `controllers/users.js` throws two different errors (NotFound vs Validation) for the two cases.



Registered e-mail, wrong password *(04_login_wrong_password.png)*



Unregistered e-mail *(05_login_unknown_email.png)*

5. Root cause analysis – BUG-01 (password overwritten on Settings save)

Bug BUG-01 is caused by a single wrong boolean operator in the `updateUser` function of `backend/controllers/user.js`. The code intends to hash and save the password only when the client sent a non-empty one, and it was written as `password !== undefined || password !== ""`. Because a value can never be both `undefined` and an empty string at the same time, at least one side is always true, so the whole condition is always true. The React Settings form always sends the `password` field, and it is an empty string when the user does not type a new one. That empty string is therefore passed to bcrypt, hashed, and stored in place of the real password hash whenever any profile field is saved. I confirmed this by testing: the original password stopped working after a bio-only save, and an empty password then returned HTTP 200 with a token. A likely reason it was missed is that a test which only checks that the update returns 200 would still pass, because the damage only shows up when someone logs in again after a profile update. The fix is to use `&&` (or simply `if (password)`), and also to reject empty passwords in the login endpoint. To prevent regressions, add an automated test that updates a profile without a password and then asserts that the old password still logs in.

6. Other observations (minor)

- Error copy has stray punctuation: "Email already exists.. try logging in", "Title already exists.. ".
- The 404 page is rendered outside the app layout, so the navbar disappears and the user only has "Go to home page" (`38_404_page_no_navbar.png`).

- Editing a title changes the article URL; the old URL immediately returns 404 (bookmarks and shared links break).
- A user can follow themselves (`POST /api/profiles/<own-name>/follow` returns 200 with `following:true`).
- Sign-up has no client-side password minimum while login does (see BUG-05); no show-password toggle or password hint anywhere.

7. Reproducing this testing

Scripts used are in `task1-qa/test-scripts/` (Playwright) with setup notes in `task1-qa/SETUP.md` .
Screenshots are in `task1-qa/screenshots/` .